

Verilog 硬體描述語言

- 函數(Function)
- 任務(Task)
- 函數呼叫函數
- 任務呼叫函數及任務
- 系統函數與任務

"function" Description (1)

- ❑ 在行為模型 (Behavioral Model) 的設計中，我們通常在程式的許多地方重複著相同描述的程式碼，這在一般的電腦程式語言裡，通常會將重複用到的程式碼設計成"函數"或是"副程式"的形式，再去引用，而不用繁瑣的鍵入同樣的程式碼；在 Verilog 中，對於相同的程式碼也可以在程式中的不同地方被引用，而不需要繁瑣的一再鍵入相同的程式碼。
- ❑ function 敘述是以關鍵字 "function" 與 "endfunction" 宣告，其中不能有「延遲時間」或「事件控制之指令」之存在，且函數僅能傳回單一值並至少要有一個輸入。
- ❑ function 敘述同樣是被包含在 Verilog 階層式的架構中，同樣可以用階層式的名稱來引用到它。
- ❑ function 的傳回值是儲存在和函數同樣名稱的這個變數中。

"function" Description (2)

// Func_AS.V: 使用函數來設計相加、加1及相減處理

```
module Func_AS (a, b, Add, Sub, Inc, Switch, c);
```

```
input  [3:0] a, b;
```

```
input  Add, Sub, Inc, Switch;
```

```
output [3:0] c;
```

```
reg    [3:0] c;
```

```
always @(a or b or Add or Sub or Inc or Switch)
```

```
begin
```

```
    if(Add)    // 相加
```

```
        c = a + b;
```

```
    else
```

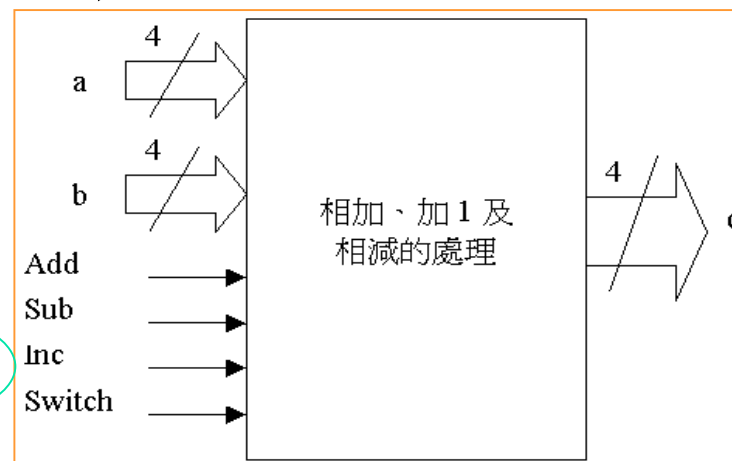
```
        if(Switch) // “0”進行交換 a, b
```

```
            c = Sub_Inc_Dec(a, b, Sub, Inc);
```

```
        else
```

```
            c = Sub_Inc_Dec(b, a, Sub, Inc);
```

```
    end
```



function

改為(a, Sub, b, Inc)後,Compiler
會成功,執行也有結果,但結果是
錯的,注意這是嚴重的錯誤。

"function" Description (3)

4-bits

```
function [3:0] Sub_Inc_Dec;
```

```
input [3:0] a, b; //對掉看看
```

```
input Sub, Inc;
```

```
begin
```

```
    if(Sub) // 相減
```

```
        Sub_Inc_Dec = (a - b);
```

```
    else
```

```
        if(Inc) // 加1
```

```
            Sub_Inc_Dec = (a + 1'b1);
```

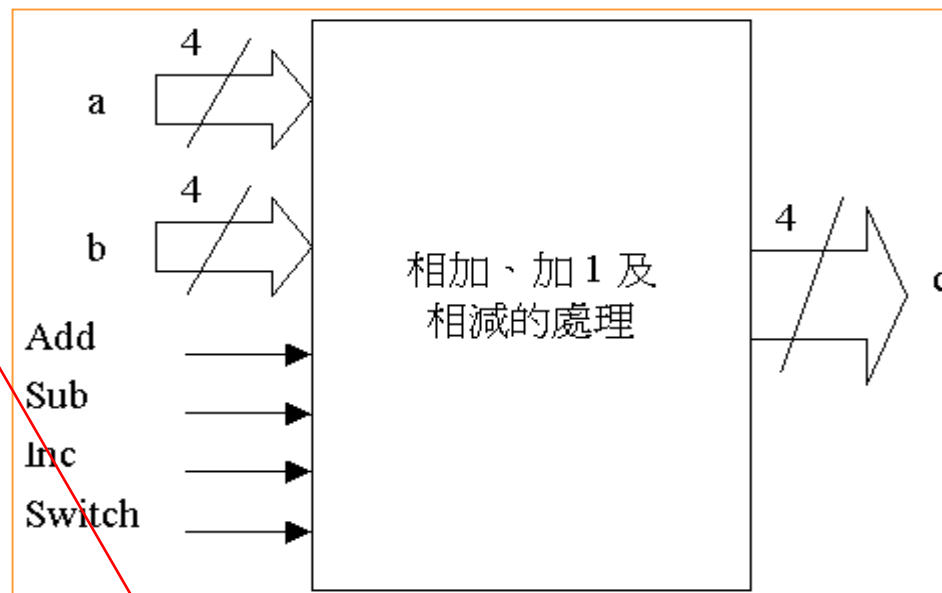
```
        else // 減1
```

```
            Sub_Inc_Dec = (a - 1'b1);
```

```
    end
```

```
endfunction
```

```
endmodule
```



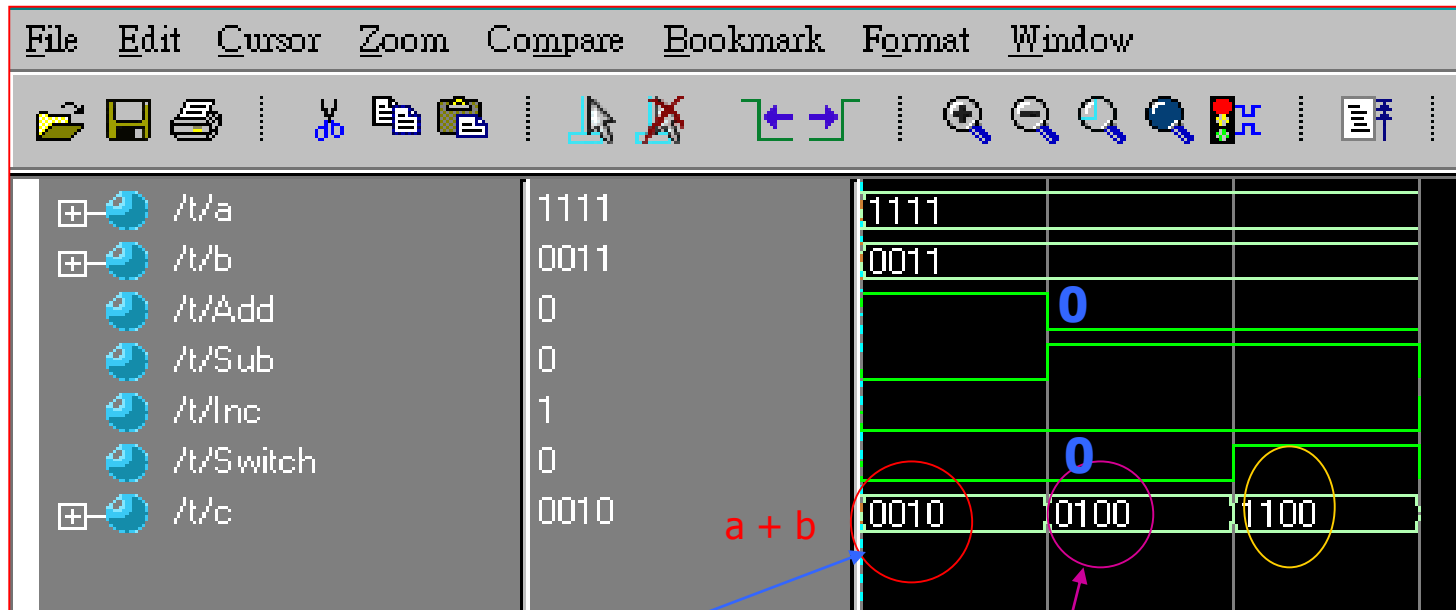
傳回值是儲存在和函數同樣名稱的這個變數中



4位元對1位元, 只取最右方的位元

Simulation Waveform

Core:2_10_FUNC_AS



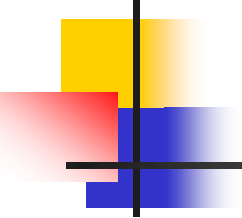
```

begin
  if(Add) // 相加
    c = a + b;
  else
    if(Switch) // "0"进行交换 a, b
      c = Sub_Inc_Dec(a, b, Sub, Inc);
    else
      c = Sub_Inc_Dec(b, a, Sub, Inc);
end
    
```

1 0

```

begin
  if(Sub) // 相减
    Sub_Inc_Dec = (a - b);
  else
    if(Inc) // 加1
      Sub_Inc_Dec = (a + 1'b1);
    else // 减1
      Sub_Inc_Dec = (a - 1'b1);
end
    
```



範例練習

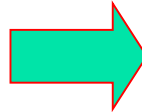
function 函數(6-001)

以8位元奇同位元產生器來
完成16位元奇同位元產生器

```
//16-bit even-parity generator using two 8-bit  
even-parity generators
```

```
module even_parity_16(Din, Pout);  
    input [15:0]Din;  
    output Pout;  
    reg [7:0]High_byte;  
    reg [7:0]Low_byte;  
    reg High, Low;  
    reg Pout;  
  
    always@(Din)  
        begin  
            High_byte = Din[15:8];  
            Low_byte = Din[7:0];  
            High = even8(High_byte);  
            Low = even8(Low_byte);  
            Pout = High^Low;  
        end  
  
    function even8;  
        input[7:0]I;  
        begin  
            even8 = ^I;  
        end  
    endfunction  
endmodule
```

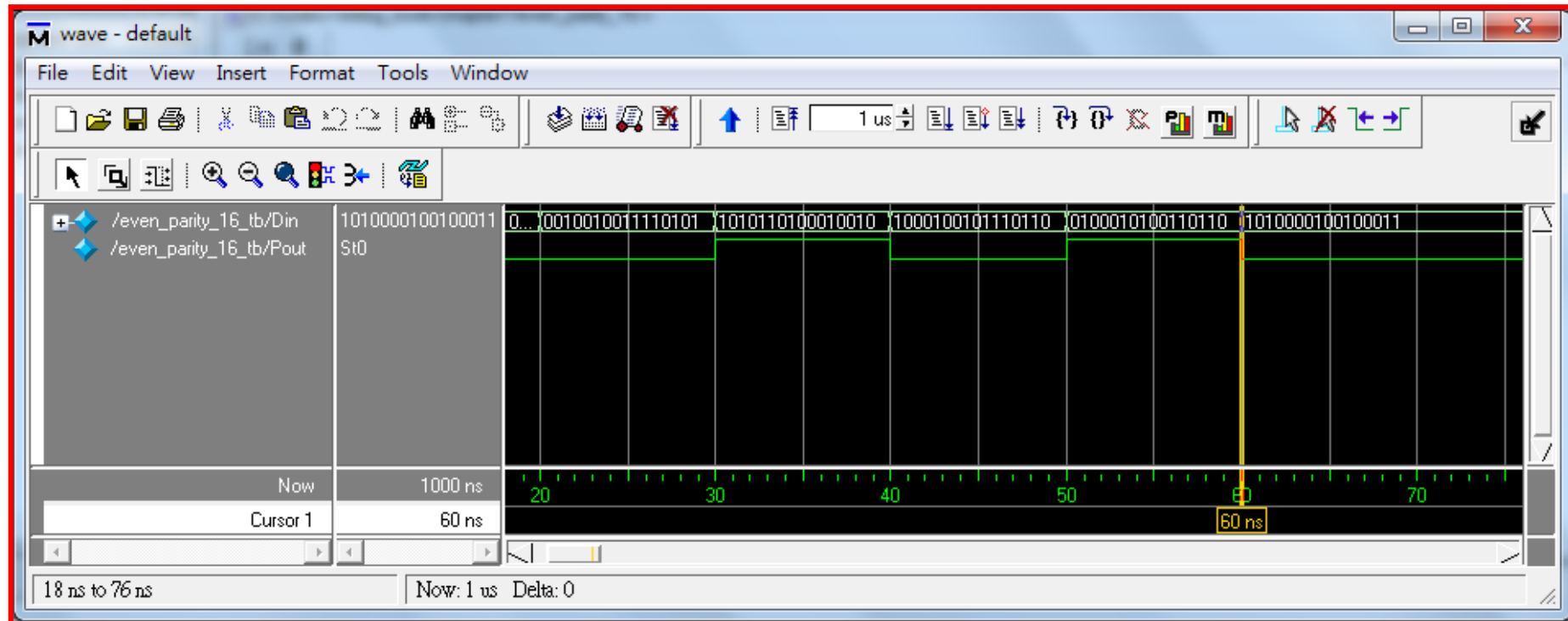
Test bench

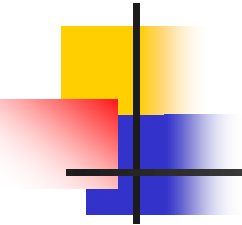


```
module even_parity_16_tb();  
    reg [15:0]Din;  
    wire Pout;  
  
    even_parity_16 even_parity_16(.Din(Din),  
                                   .Pout(Pout));  
  
    initial  
        begin  
            #0 Din = 16'h0;  
            #10 Din = 16'h0a0a;  
            #10 Din = 16'h24f5;  
            #10 Din = 16'had12;  
            #10 Din = 16'h8976;  
            #10 Din = 16'h4536;  
            #10 Din = 16'ha123;  
        end  
  
endmodule
```

Simulation Waveform

Code:6_001





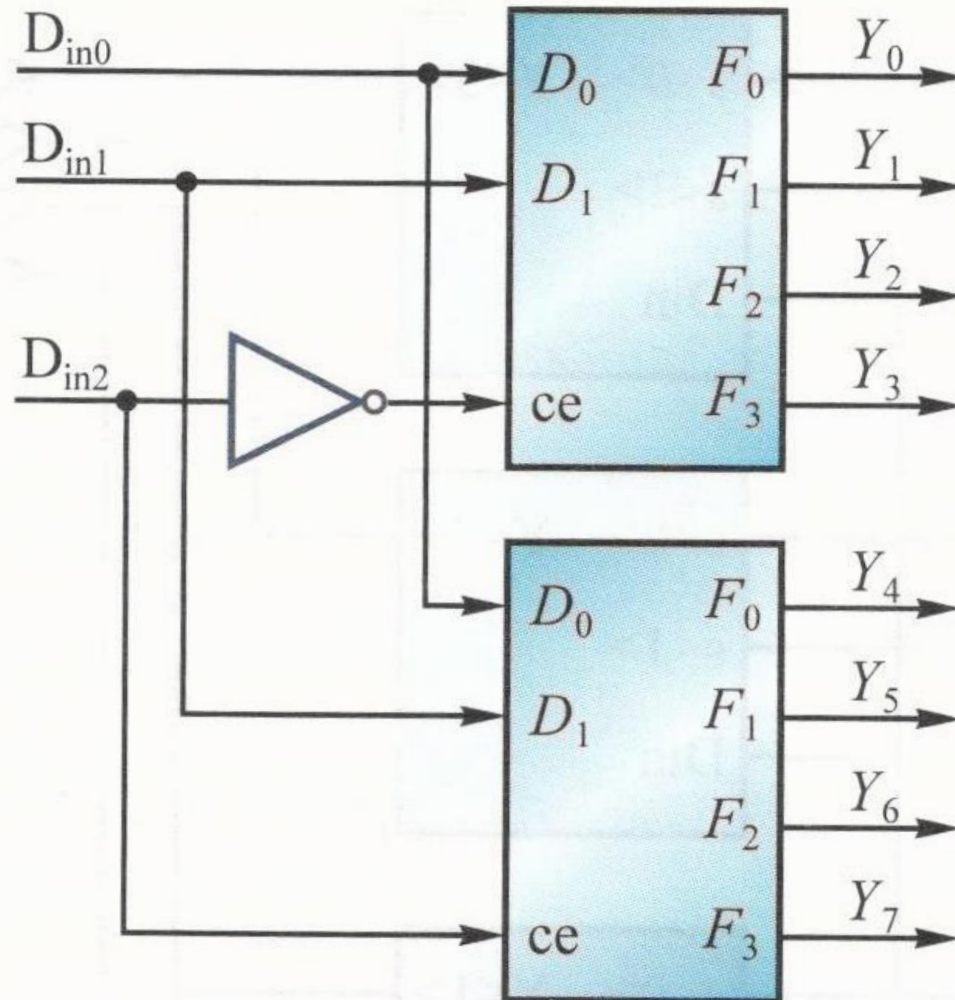
範例練習

function 函數(6-002)

以2對4解碼器

來產生3對8解碼器

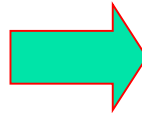
3 to 8 Decoder Block Diagram



```
//3 to 8 decoder using a function of 2 to 4 decoders
module decod_fcn(Din, Y);
    input [2:0]Din;
    output [7:0]Y;
    reg [7:0]Ytmp;
    always@(Din)
        begin
            Ytmp[7:4] = decod4(Din[1:0], Din[2]);
            Ytmp[3:0] = decod4(Din[1:0], ~Din[2]);
        end
    assign Y = Ytmp;

    function [3:0]decod4;
        input [1:0]D;
        input ce;
        if(ce == 1'b1)
            begin
                case(D)
                    2'b00:decod4 = 4'b0001;
                    2'b01:decod4 = 4'b0010;
                    2'b10:decod4 = 4'b0100;
                    2'b11:decod4 = 4'b1000;
                    default:decod4 = 4'b0000;
                endcase
            end
        else
            decod4 = 4'b0000;
        endfunction
endmodule
```

Test bench



```
module decod_fcn_tb();

    reg [2:0]Din; //input
    wire [7:0]Y;
    reg clk;

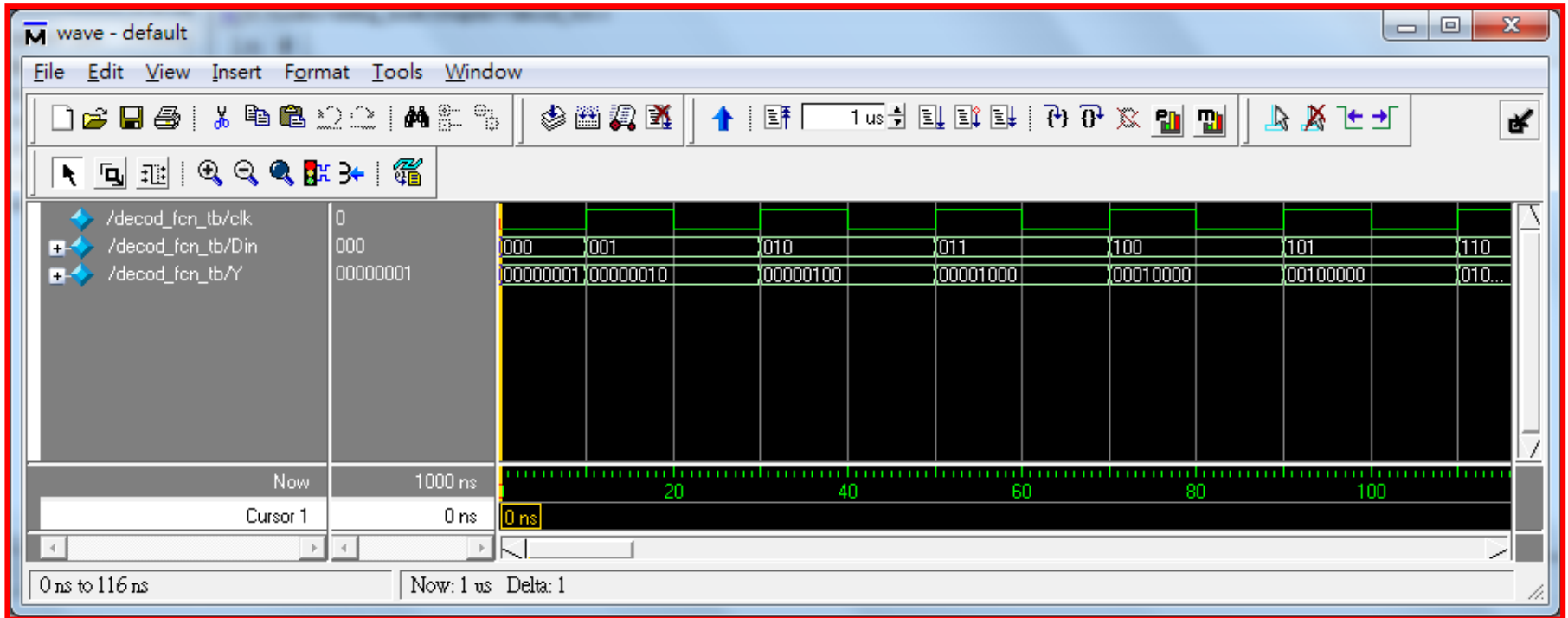
    decod_fcn decod_fcn(.Din(Din),
                        .Y(Y));

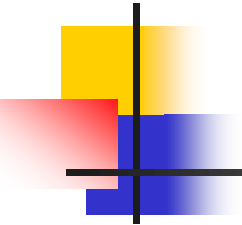
    initial
        begin
            #0 Din = 3'b0;
            clk = 1'b0;
        end

    always #10 clk = ~clk;

    always@(posedge clk)
        begin
            if(Din < 3'b111)
                Din = Din + 1;
            else
                Din = 3'b000;
        end
    endmodule
```

Code:6_002





Verilog 硬體描述語言

- 函數(Function)
- 任務(Task)
- 函數呼叫函數
- 任務呼叫函數及任務
- 系統函數與任務

“task” Description (1)

- ❑ Task(任務)敘述與 function (函數)敘述大致相同，都是將模組中重複用到的程式碼設計成函數或是副程式的形式，再去引用，而不用繁瑣的鍵入同樣的程式碼。
- ❑ function 敘述至少要有一個或以上的 input 宣告；而 task 敘述可以擁有零個或是更多的 input、output 或是 inout 宣告。
- ❑ task 敘述是以關鍵字 “task” 與 “endtask” 宣告。
- ❑ task 敘述如同程式語言的 Procedure (程序或步驟)。
- ❑ task 敘述同樣可以用階層式的名稱來引用到它。
- ❑ function 與 task 敘述都可以擁有本身的區域變數與暫存器，但不能擁有 wire 型態的變數。
- ❑ function 與 task 敘述只能用於行為模型 (Behavioral Model) 的敘述中，且本身不能有 always 的敘述，但是通常是在一個 always 區塊中被引用。

函數 (function)	任務 (task)
● function 可以引用其他的 function，但不能引用其他的 task。 ● function 至少要有一個或以上的 input 宣告。	● task 可以引用其他的 task 與 function。 ● task 可以擁有零個或是更多的 input、output 或是 inout 宣告。

“task” Description (2)

//範例：Sort4.V, 由小到大排序4筆資料 (結果: $ra \leq rb \leq rc \leq rd$)

// You may distribute freely, as long as this header remains attached.

```
module Sort_4_Data(a, b, c, d, ra, rb, rc, rd);
```

```
parameter size = 4 - 1;
```

```
input [size:0] a, b, c, d;
```

```
output [size:0] ra, rb, rc, rd;
```

```
reg [size:0] ra, rb, rc, rd;
```

```
always @(a or b or c or d)
```

```
begin :Label_Name
```

```
// 少了:Label_Name 會發生: Expecting
```

```
// statement 的錯誤訊息
```

```
reg [size:0] va, vb, vc, vd;
```

```
{va,vb,vc,vd} = {a, b, c, d};
```

```
Sort_2_Data(va, vc);
```

```
Sort_2_Data(vb, vd);
```

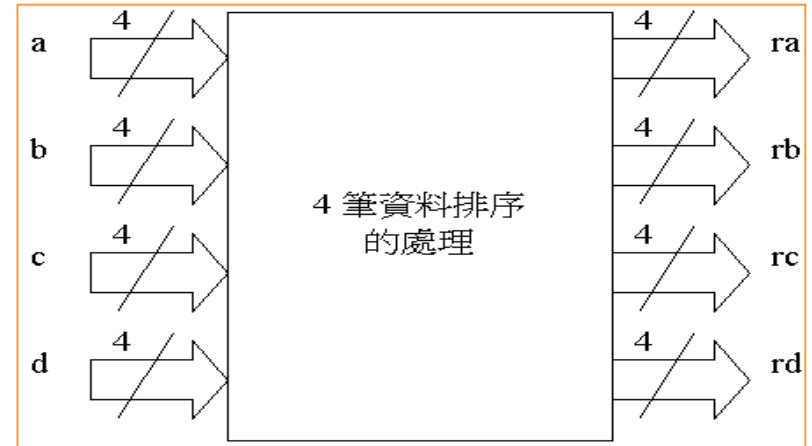
```
Sort_2_Data(va, vb);
```

```
Sort_2_Data(vc, vd);
```

```
Sort_2_Data(vb, vc);
```

```
{ra, rb, rc, rd} = {va, vb, vc, vd};
```

```
end
```



使用Local Variable時，需對begin....end命名

宣告Local Variable，且僅可在begin.....end內使用

兩兩比大小

(3,1,4,2) → (3,1,4,2) → (3,1,4,2)
→ (1,3,4,2) → (1,3,2,4) → (1,2,3,4)

“task” Description (3)

```
task Sort_2_Data; ←
```

改用Function功能

```
  inout [size:0] x, y; ←
```

改用Input 或 Output功能

```
  reg [size:0] temp;
```

輸入同時具有輸出功能

```
    if (x > y)
```

```
    begin // 交換 (Swap)
```

```
      temp = x;
```

```
      x = y;
```

```
      y = temp;
```

比較大的往後排

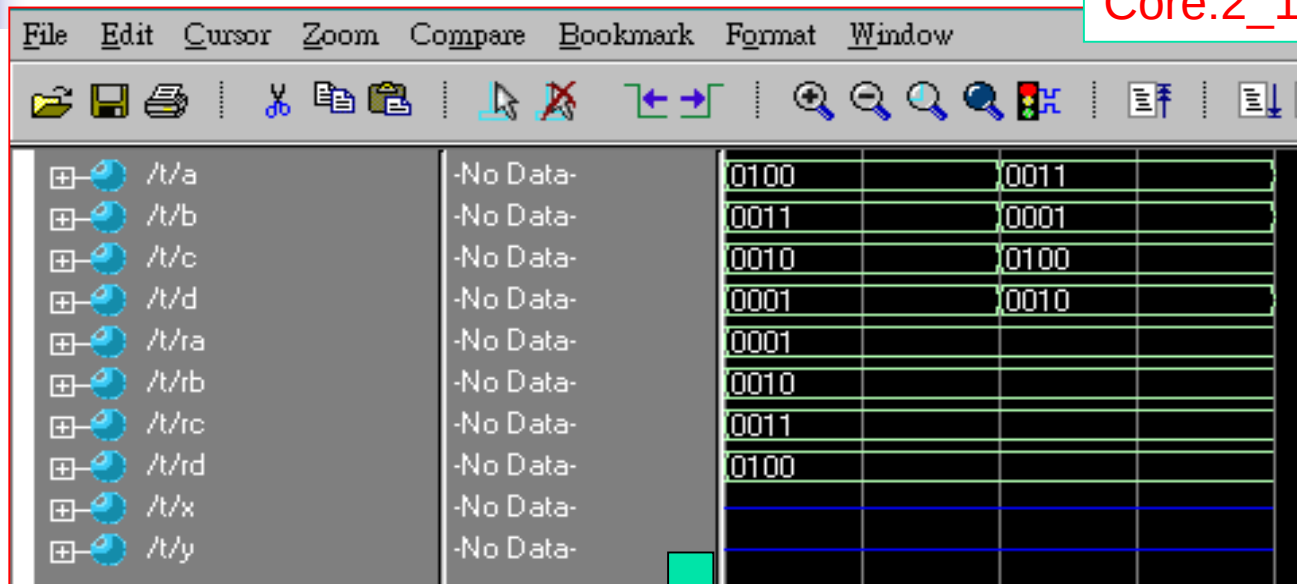
```
    end
```

```
  endtask
```

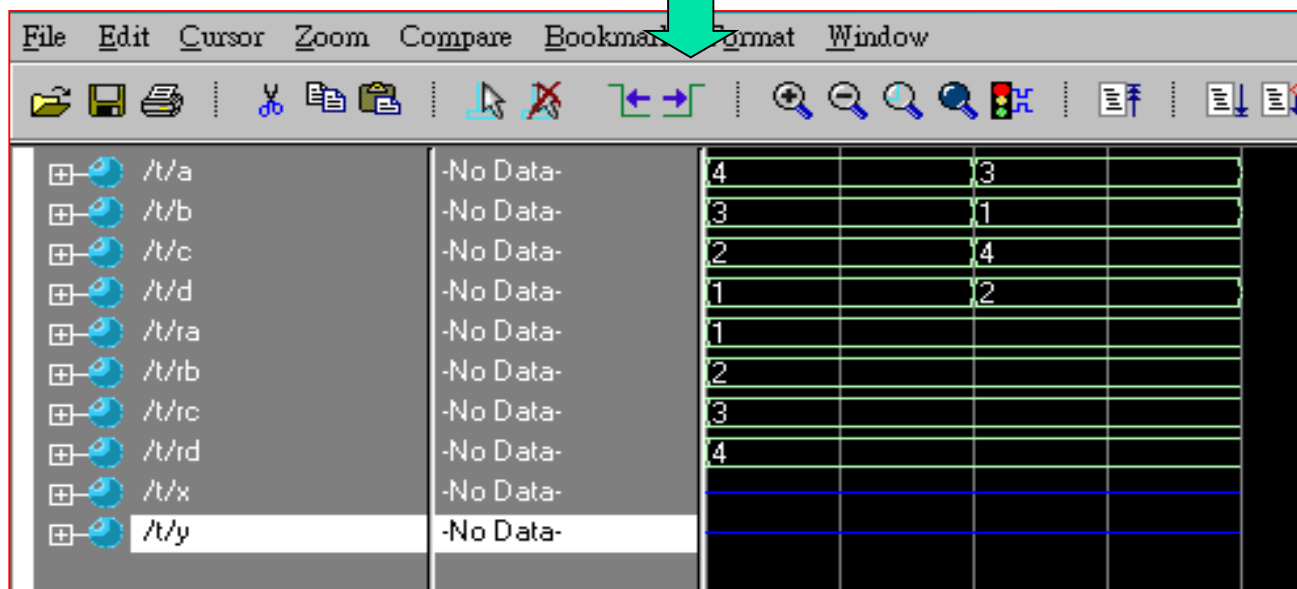
```
endmodule
```

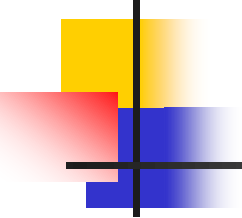
Simulation Waveform

Core:2_12_SORT4



練習：
由大到小
的方法





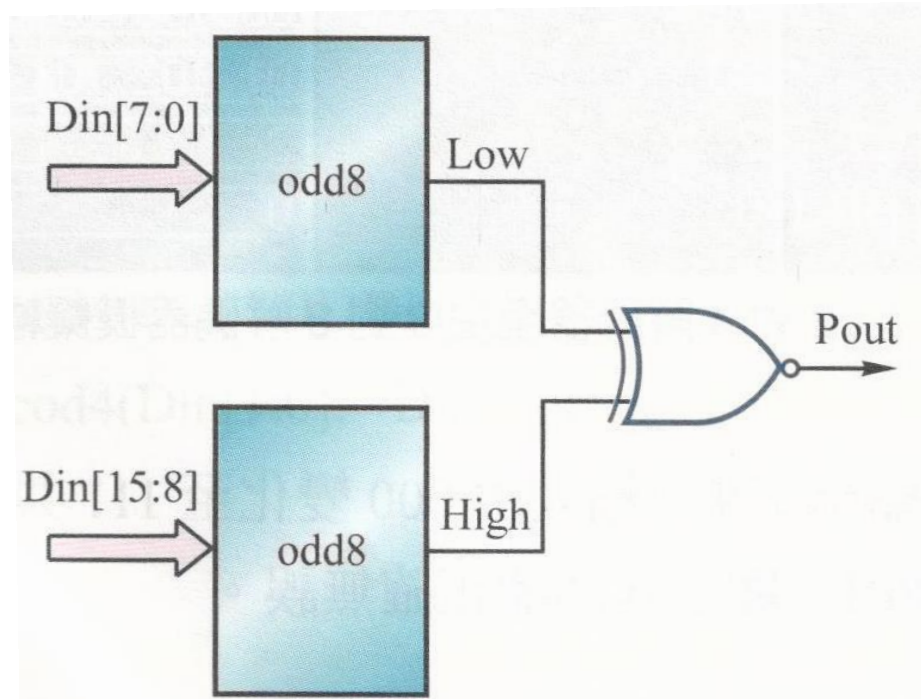
範例練習

Task函數(6-003)

以8位元奇同位元產生器來完成

16位元偶同位元產生器

以8位元奇同位完成16位元奇同位產生器方塊圖



- ❑ 輸入16位元資料的高位元組與低位元組分別送到獨立的8位元奇同位元產生器，接著將該二個8位元奇同位輸出位元經過XOR閘，產生16位元之奇同位元。
- ❑ 在程式中，High_byte及Low_byte分別接收輸入資料之高位元經之奇同位High及Low，再透過XNOR運算子產生16位元奇同位元Pout。

```
//16-bit odd-parity generator using two 8-bit  
odd-parity generators
```

```
//ex7.5
```

```
module odd_parity_16(Din, Pout);
```

```
    input [15:0]Din;
```

```
    output Pout;
```

```
    reg [7:0]High_byte;
```

```
    reg [7:0]Low_byte;
```

```
    reg High, Low;
```

```
    reg Pout;
```

```
    always@(Din)
```

```
        begin
```

```
            High_byte = Din[15:8];
```

```
            Low_byte = Din[7:0];
```

```
            odd8(High_byte, High);
```

```
            odd8(Low_byte, Low);
```

```
            Pout = High~^Low;
```

```
        end
```

Task 函數 (6-003)

```
task odd8;
```

```
    input [7:0]I;
```

```
    output odd8;
```

```
    begin
```

```
        odd8 = ~^I;
```

```
    end
```

```
endtask
```

```
endmodule
```

```
module odd_parity_16_tb();  
    reg [15:0]Din;  
    wire Pout;  
  
    odd_parity_16 odd_parity_16(.Din(Din),  
                                .Pout(Pout)  
                                );
```

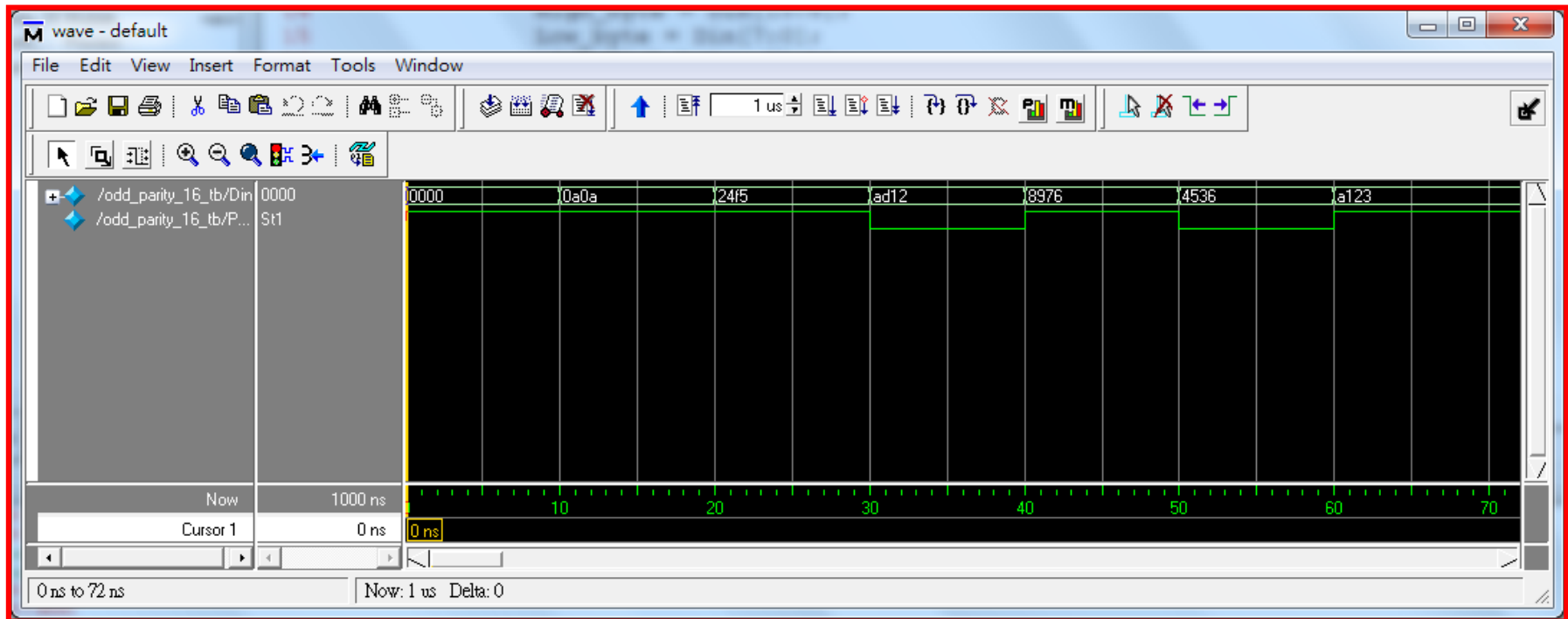
```
    initial  
        begin  
            #0  Din = 16'b0;  
            #10 Din = 16'h0a0a;  
            #10 Din = 16'h24f5;  
            #10 Din = 16'had12;  
            #10 Din = 16'h8976;  
            #10 Din = 16'h4536;  
            #10 Din = 16'ha123;  
        end
```

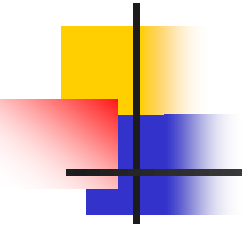
```
endmodule
```

Task 函數(6-003)
- Test bench

Simulation Waveform

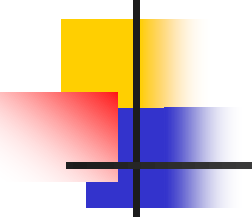
Code:6_003





Verilog 硬體描述語言

- 函數(Function)
- 任務(Task)
- 函數呼叫函數
- 任務呼叫函數及任務
- 系統函數與任務

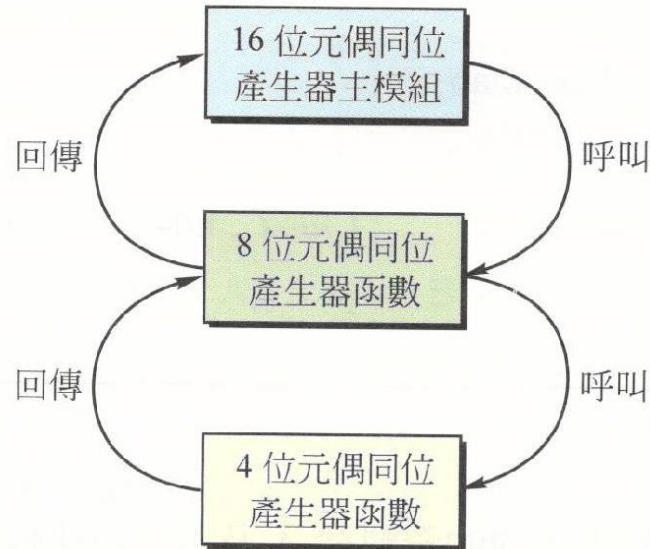


範例練習

函數呼叫函數(6-004)

以函數呼叫函數來完成
16位元偶同位元產生器

函數呼叫函數的程序圖



- ❑ 在模組 `even16_fun_fun.v` 中，輸入 16 位元資料分為二組送至 `High_byte` 及 `Low_byte`，並傳至函數 `even8` 以產生 8 位元之偶同位元；再回傳至 `High` 及 `Low` 作互斥或運算，最後送至輸出 `Pout`。
- ❑ 8 位元偶同位函數將所收到的 8 位元資料細分為高 4 位元及低 4 位元二組，再分別呼叫 4 位元偶同位產生器函數，以產生偶同位。
- ❑ 在 4 位元偶同位產生器函數 `even4` 中，會將所取得的 4 位元以互斥或指令產生偶同位並傳回至上一層函數。
- ❑ 最後，以巢狀式函數呼叫，完成 16 位元偶同位產生器電路。

//16-bit even-parity generator modified from ex7.2 using function call

//ex7.7

```
module even16_fun_fun(Din, Pout);
```

```
    input [15:0]Din;
```

```
    output Pout;
```

```
    reg [7:0]High_byte;
```

```
    reg [7:0]Low_byte;
```

```
    reg High, Low;
```

```
    reg Pout;
```

```
    always@(Din)
```

```
        begin
```

```
            High_byte = Din[15:8];
```

```
            Low_byte = Din[7:0];
```

```
            High = even8(High_byte);
```

```
            Low = even8(Low_byte);
```

```
            Pout = High^Low;
```

```
        end
```



(Code:6-004)

```
function even8;
```

```
    input[7:0]I8;
```

```
    begin
```

```
        even8 = even4(I8[7:4])^even4(I8[3:0]);
```

```
    end
```

```
endfunction
```

```
function even4;
```

```
    input[3:0]I4;
```

```
    begin
```

```
        even4 = ^I4;
```

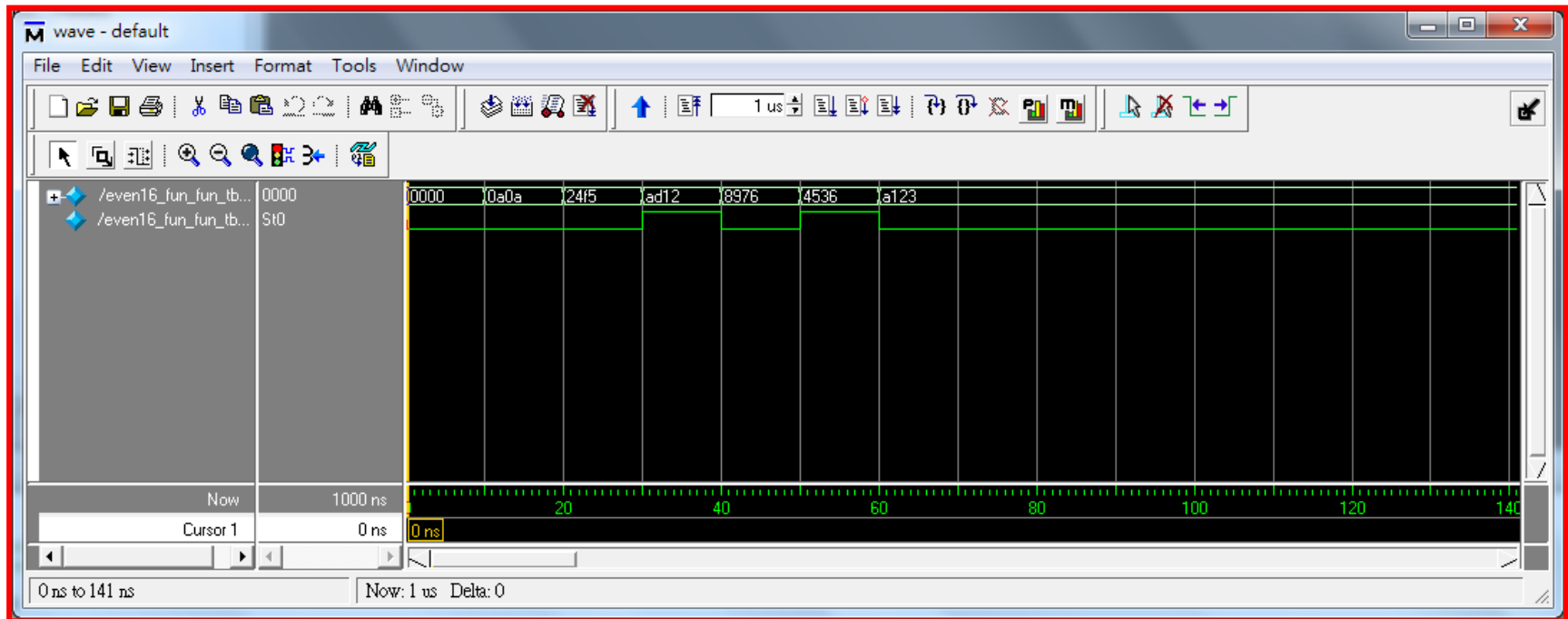
```
    end
```

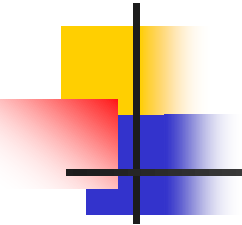
```
endfunction
```

```
endmodule
```

Simulation Waveform

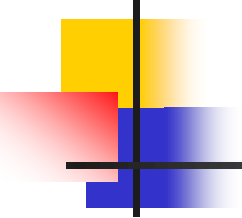
Code:6_004





Verilog 硬體描述語言

- 函數(Function)
- 任務(Task)
- 函數呼叫函數
- 任務呼叫函數及任務
- 系統函數與任務

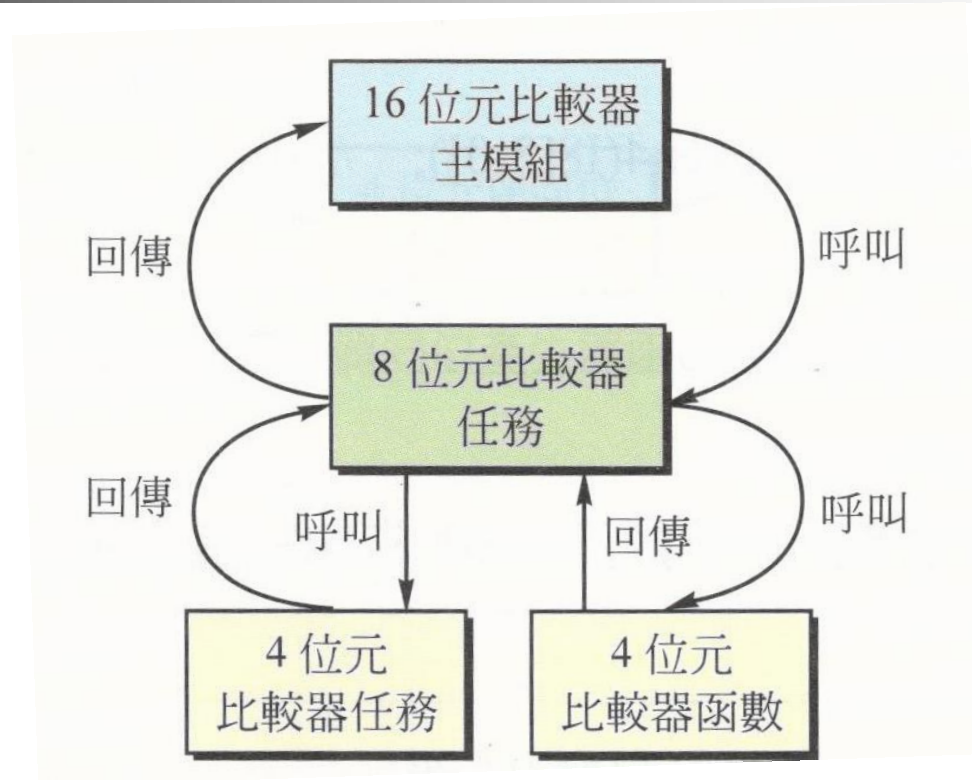


範例練習

任務呼叫函數與任務(6-005)

以任務呼叫函數與任務
來完成16位元比較器

任務呼叫函數與任務的程序圖



- ❑ 主要模組comp16_t_tf.v分別呼叫任務compre8二次，以完成16位元資料的比較工作；而任務compare8再分別呼叫任務compare4_T與函數compare4_F，各完成4位元資料之比較後，以組成8位元資料的比較工作。

```
//16-bit Coparator using 8-bit-comparator task  
//to call a task and a function with 4-bit-comparator  
//ex7.8
```

```
module comp16_t_tf(A, B, Yout);  
    input [15:0]A;  
    input [15:0]B;  
    output [2:0]Yout;  
    reg [7:0]A8bit;  
    reg [7:0]B8bit;  
    reg [2:0]Yout;  
    always@(A or B)  
        begin  
            A8bit = A[15:8];  
            B8bit = B[15:8];  
            compare8(A8bit, B8bit, Yout);  
            if(Yout == 3'b010)//A[15:8]=B[15:8]  
                begin  
                    A8bit = A[7:0];  
                    B8bit = B[7:0];  
                    compare8(A8bit, B8bit, Yout);  
                end  
        end  
end
```

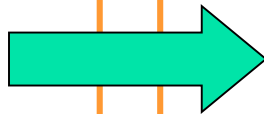
(Code:6-005)

```

task compare8;
    input [7:0]I_A;
    input [7:0]I_B;
    output [2:0]S;
    reg [3:0]A4bit;
    reg [3:0]B4bit;

    begin
        A4bit = I_A[7:4];
        B4bit = I_B[7:4];
        compare4_T(A4bit, B4bit, S);
        if(S == 3'b010) //I_A[7:4]==I_B[7:4]
            begin
                A4bit = I_A[3:0];
                B4bit = I_B[3:0];
                S=compare4_F(A4bit, B4bit);
            end
        end
    end
endtask

```



```

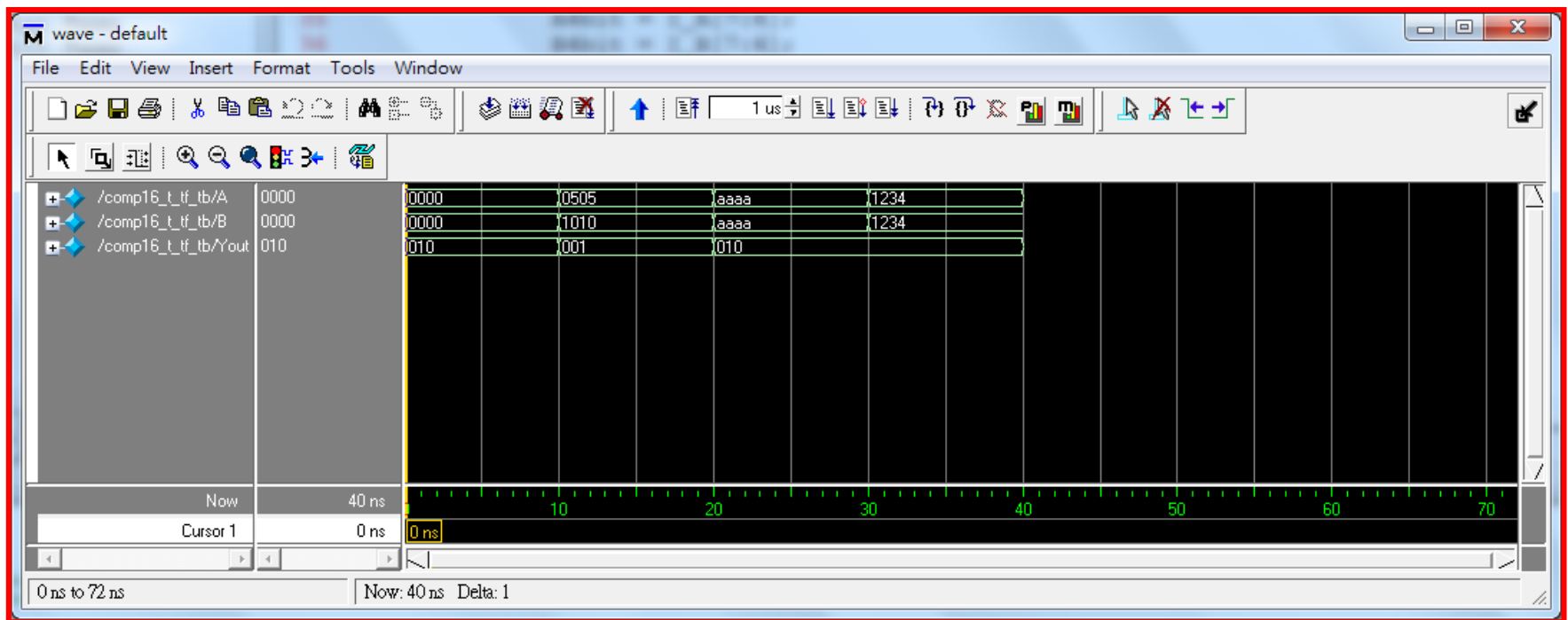
task compare4_T;
    input [7:4]I_A;
    input [7:4]I_B;
    output [2:0]S;
    begin
        if(I_A > I_B)
            S = 3'b100;
        else if(I_A == I_B)
            S = 3'b010;
        else
            S = 3'b001;
        end
    endtask

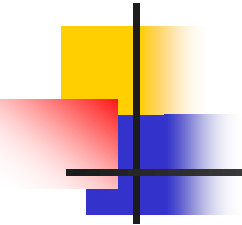
function [2:0]compare4_F;
    input [3:0]I_A;
    input [3:0]I_B;
    begin
        if(I_A > I_B)
            compare4_F = 3'b100;
        else if(I_A == I_B)
            compare4_F = 3'b010;
        else
            compare4_F = 3'b001;
        end
    endfunction
endmodule

```

Simulation Waveform

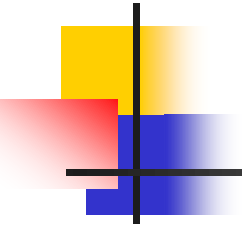
Code:6_005





Verilog 硬體描述語言

- 函數(Function)
- 任務(Task)
- 函數呼叫函數
- 任務呼叫函數及任務
- 系統函數與任務



系統函數與任務

- ❑ 系統函數與任務是以符號(\$)所引導的關鍵字。
- ❑ 並非所有的系統函數與任務均可被ModelSim系統所接受。
- ❑ \$display(<系統函數>, “字串與格式”, <輸出入埠名稱>)
 - 顯示後跳下一行
- ❑ \$write(<系統函數>, “字串與格式”, <輸出入埠名稱>)
 - 顯示後游標在同一行
- ❑ \$finish
 - 跳離執行中的模擬程式